

Article

Improved Bidirectional RRT* Algorithm for Robot Path Planning

Peng Xin, Xiaomin Wang *, Xiaoli Liu, Yanhui Wang , Zhibo Zhai and Xiqing Ma

Key Laboratory of Intelligent Industrial Equipment Technology of Hebei Province, School of Mechanical and Equipment Engineering, Hebei University of Engineering, Handan 056038, China

* Correspondence: ztds8579390@163.com

Abstract: In order to address the shortcomings of the traditional bidirectional RRT* algorithm, such as its high degree of randomness, low search efficiency, and the many inflection points in the planned path, we institute improvements in the following directions. Firstly, to address the problem of the high degree of randomness in the process of random tree expansion, the expansion direction of the random tree growing at the starting point is constrained by the improved artificial potential field method; thus, the random tree grows towards the target point. Secondly, the random tree sampling point grown at the target point is biased to the random number sampling point grown at the starting point. Finally, the path planned by the improved bidirectional RRT* algorithm is optimized by extracting key points. Simulation experiments show that compared with the traditional A*, the traditional RRT, and the traditional bidirectional RRT*, the improved bidirectional RRT* algorithm has a shorter path length, higher path-planning efficiency, and fewer inflection points. The optimized path is segmented using the dynamic window method according to the key points. The path planned by the fusion algorithm in a complex environment is smoother and allows for excellent avoidance of temporary obstacles.

Keywords: artificial potential field method; improved bidirectional RRT* algorithm; dynamic window method; fusion algorithm; temporary obstacles avoidance



Citation: Xin, P.; Wang, X.; Liu, X.; Wang, Y.; Zhai, Z.; Ma, X. Improved Bidirectional RRT* Algorithm for Robot Path Planning. *Sensors* **2023**, *23*, 1041. <https://doi.org/10.3390/s23021041>

Academic Editors: Luis Payá, Oscar Reinoso García and Helder Jesus Araújo

Received: 22 November 2022

Revised: 31 December 2022

Accepted: 5 January 2023

Published: 16 January 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Mobile robots have a wide range of applications in industry, agriculture, and daily life. To enable robots to quickly reach their target point and complete a designated task in a complex environment, an excellent path-planning algorithm should be employed to plan an effective path. Therefore, a path-planning algorithm is a core facet of the development of mobile robots [1,2]. Common path-planning algorithms include the artificial potential field method [3], ant colony algorithms [4], the A* algorithm [5], the dynamic window method [6], etc.

The RRT algorithm is a classical path-planning algorithm [7]. The main idea of the algorithm is to find an effective path by growing a random tree continuously until the target point is also on the random tree. The defects of the RRT algorithm are obvious. The RRT algorithm randomly obtains sampling points when looking for a path, which makes its search efficiency low, and there are many redundant nodes and redundant sections in the planned path. RRT* [8] and bidirectional RRT* are commonly used to improve the RRT algorithm. At present, scholars have carried out in-depth research on the RRT algorithm. Mashayekh et al. [9] proposed Informed RRT*-Connect, which improved the sampling of RRT*-Connect and provided better solutions. In [10], the author proposes EB-RRT, which facilitates a robot's movement in dynamic environments using heuristics to plan a global path and the EB method to optimize the heuristic path. In [11], the authors propose a new rewiring method based on triangulation to improve the RRT algorithm. The planning time is closer to the optimum than the traditional RRT algorithm. In [12], a regression mechanism is added to RRT to prevent over-searching. The algorithm uses adaptive expansion to avoid duplicate searches and improve search efficiency by refining

the boundary nodes. In [13], the RRT*-AB algorithm is proposed. This algorithm can quickly determine the target area, adjust bounds, and improve computational efficiency. Qi [14] proposes the MOD-RRT* algorithm for unknown dynamic environments. The path is first planned using the improved RRT* algorithm; then, the initial path is optimized. Wang [15] used a convolutional neural network to optimize the RRT* algorithm. The prediction of optimal paths by CNN models guides the RRT* algorithm's sampling, thus improving path-planning efficiency. The authors of [16] propose an LM-RRT algorithm that uses reinforcement learning to solve path planning in narrow environments. Qie [17] proposed the (TF-RRT*) method. This algorithm only allows the sampling points to be biased towards the target point, and the random numbers grow more biased towards the target point, which effectively improves path-planning efficiency.

The authors of [18] improved search efficiency by adjusting the direction of growth tree expansion with situational data. These algorithms generally require adjustments and place restrictions on the search direction of the random tree and the selection of sampling points.

In this paper, we propose a new algorithm combining the bidirectional RRT* algorithm and the artificial potential field method, which can accelerate the combination of two trees during the sampling process and thus effectively improve path-planning efficiency. Combining the new algorithm with the dynamic window method in the global path enables a robot to reach a target point safely and avoid temporary obstacles.

2. Relate Work

2.1. Principle of RRT*

The main aim of RRT is to grow a random tree at a starting point and then grow a random point in blank space. The process transpires as follows: find the point in the random tree that is nearest to the random point, and then extend one step from the nearest point to the random point. When the extended step does not lead to successful navigation through the obstacles, new nodes are successively added to the random tree until the target point is also in the random tree and the path is found; consequently, the path-planning process is completed. Although the RRT algorithm can basically plan an effective path, its search efficiency will be greatly reduced due to the approximate uniform expansion of the random tree in the surrounding space.

The RRT* algorithm improves the process of re-selecting the parent node on the basis of the RRT algorithm. By constantly adjusting the parent node, the path is gradually optimized. This path optimization procedure will increase the path-planning time of the RRT* algorithm. The specific process is shown in Figure 1.

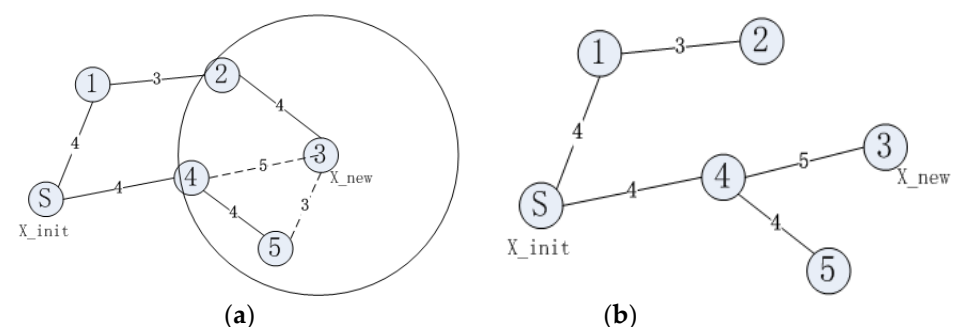


Figure 1. Principle of RRT* extension (a) Path planned by traditional RRT algorithm, (b) Optimized path.

Figure 1a shows the path planned by the RRT algorithm and Figure 1b shows the path planned by the RRT* algorithm. In Figure 1, node ③ is x_{new} and the path planned by the RRT algorithm is $S \rightarrow ① \rightarrow ② \rightarrow ③$. With node ③ as the center of the circle, a circle is drawn with a certain distance as the radius; nodes ②, ④ and ⑤ are within the circle. When comparing path $S \rightarrow ① \rightarrow ② \rightarrow ③$ and path $S \rightarrow ④ \rightarrow ③$, path $S \rightarrow ④ \rightarrow ③$ has the shortest

path length. At this stage, the path is replanned, with node ④ as the parent of node ③; the replanned path is shown in Figure 1b.

2.2. Principle of Bidirectional RRT*

When using a single random tree to search the whole space, the search efficiency is low. Using the RRT* algorithm to expand the random tree at both the start and target side can effectively improve search efficiency. When two trees are connected or when two trees are within a certain distance of each other, the search is completed, as shown in Figure 2.

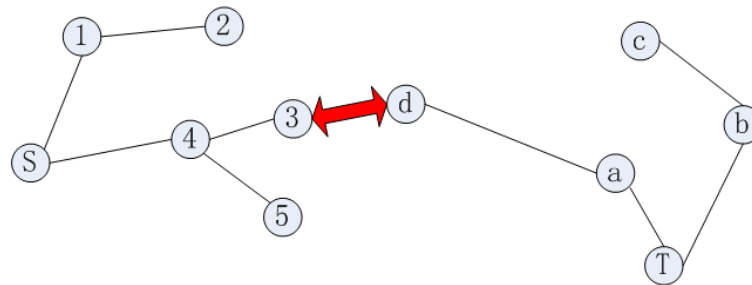


Figure 2. Principle of bidirectional RRT* extension.

In Figure 2, the red line indicates the connection of the two trees growing from the start point and target point, indicating that the global valid path has been found. The global path is as follows: S->④->③->④->④->T.

3. Improved Bidirectional RRT* Algorithm

Although the bidirectional RRT* algorithm has been greatly improved compared to the traditional RRT algorithm, there are still some problems during the expansion of the two trees, such as the excessively high degree of randomness of the sampling points, the low path-planning efficiency, and the many inflection points and redundant sections in the planned path. Therefore, the following improvements are proposed for the bidirectional RRT* algorithm.

3.1. Adding Artificial Potential Field Ideas

Although the traditional bi-directional RRT* algorithm grows random trees at both the start and target sides, the efficiency of its path planning is still low due to the random sampling points. Thus, the artificial potential field bias is increased to the first tree expansion, making the sampling more directional and improving search efficiency.

The artificial potential field gravitational function is as follows:

$$U_{att} = \frac{1}{2} \xi \rho^2(P, P_{goal}) \quad (1)$$

In Equation (1), ξ is the gravitational coefficient and $\rho(P, P_{goal})$ is the distance between the current point and the target point.

The repulsion function is as follows:

$$U_{rep} = \begin{cases} \frac{1}{2}\mu(\frac{1}{\rho(P, P_{obs})} - \frac{1}{\rho_0})^2, & \rho(P, P_{obs}) \leq \rho_0 \\ 0 & , \rho(P, P_{obs}) > \rho_0 \end{cases} \quad (2)$$

In Equation (2), μ is the repulsion coefficient and $\rho(P, P_{obs})$ is the distance between the current point and obstacle. ρ_0 is the influence range of the obstacle. Beyond this range, the obstacle has no repulsive effect on the current node.

Let the combined potential fields be

$$U = U_{att} + U_{rep} \quad (3)$$

When the traditional artificial potential field method is used to calculate the joint potential field of the nodes around the current position, the search range is too small, which is not conducive to the determination of the surrounding nodes. Therefore, an extended search range is implemented. If the search range is too large in a complex environment, it can reduce the efficiency of path planning; therefore, the search range was changed to 5×5 , as shown in Figure 3.

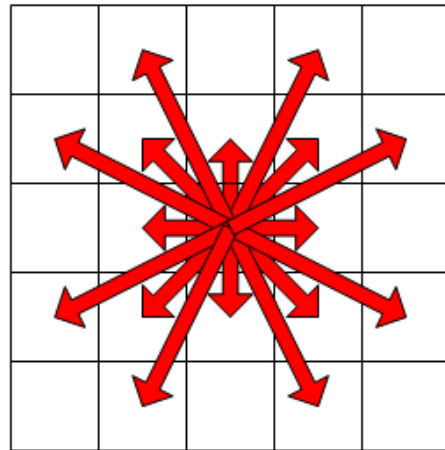


Figure 3. 5×5 search range.

The 5×5 search range artificial potential field method was added to the selection of sampling points, as shown in Figure 4.

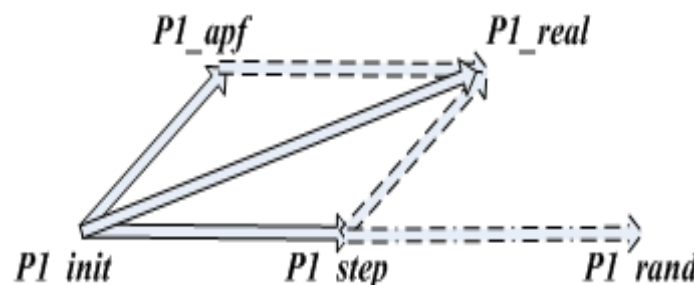


Figure 4. Sampling with the addition of artificial potential field concepts.

In Figure 4, $P1_init$ is the current node, $P1_APF$ is the artificial potential field method's bias of the current node, $P1_rand$ is the randomly sampled node, $P1_step$ is the node extended by one step, and $P1_real$ is the actual extended node. The artificial potential field method is first used to improve the sampling points; then, the nodes in the path are re-selected as parents to further optimize the path.

3.2. Adjusting the Sampling Direction of a Random Tree Growing at a Target Point

The random tree growing at the target side is biased towards the sampling point of the starting tree when it is expanded, as shown in Figure 5.

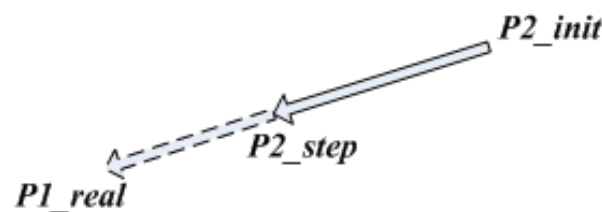


Figure 5. The sampling direction of a random tree growing at target point.

In Figure 5, $P1_{real}$ is the actual sampling point of the random tree grown at the starting point, $P2_{init}$ is the closest point to $P1_{real}$, which is growing at the end point. $P2_{step}$ is an extension of one step towards $P2_{init}$ to $P1_{real}$. The extension of the random tree growing at the end point is deflected to the actual sampling point of the random tree growing at the starting point, thereby accelerating the connection of the two trees and improving path-planning efficiency.

The improved bidirectional RRT* algorithm was compared with the traditional bidirectional RRT* algorithm in an obstacle-free and an obstacle-strewn environment to verify the effectiveness of the improvements. A comparison of the two algorithms in the obstacle-free environment is shown in Figure 6.

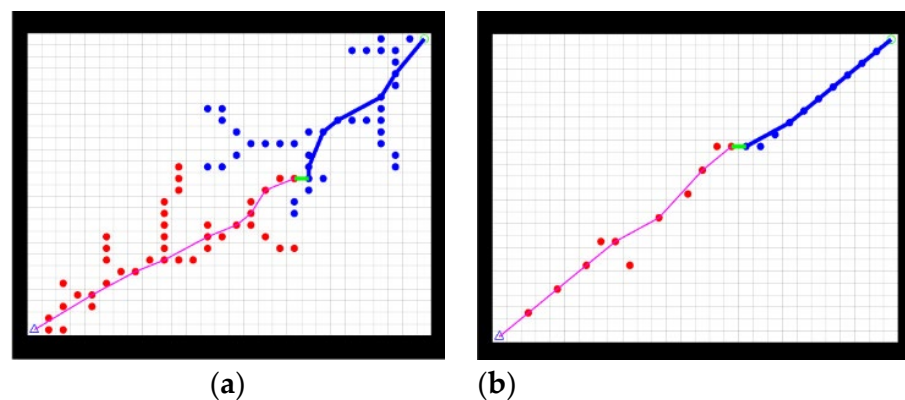


Figure 6. Comparison of two algorithms in obstacle-free environment. (a) Traditional bidirectional RRT* algorithm, (b) Improved bidirectional RRT* algorithm.

Figure 6a shows the path planned by the bidirectional RRT* in the obstacle-free environment and Figure 6b shows the path planned by the improved bidirectional RRT*. By comparing Figure 6a,b, it is evident that the improved bidirectional RRT* planning results in significantly shorter paths, smoother paths, and fewer sampling points in the obstacle-free environment.

When there are obstacles in the test environment, if the actual sampling point is on the obstacle, the point is dropped, and the search is repeated. Figure 7 shows a comparison of the two algorithms when an obstacle is present in the environment.

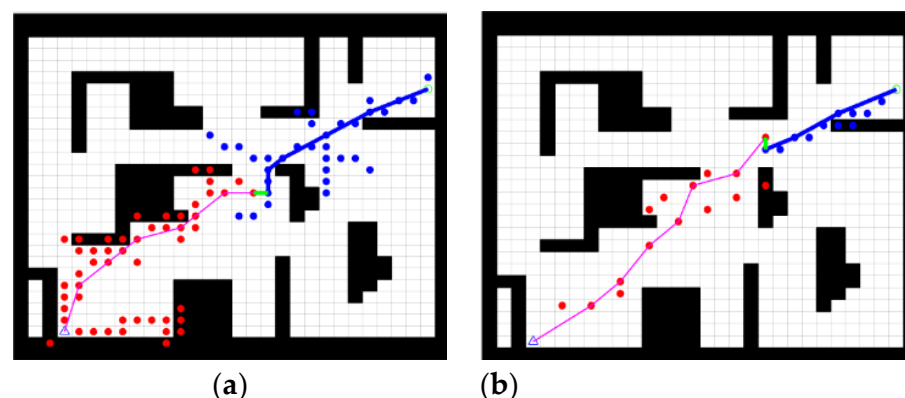


Figure 7. Comparison of two algorithms in obstacle-strewn environment. (a) Traditional bidirectional RRT* algorithm, (b) Improved bidirectional RRT* algorithm.

Figure 7a shows the path planned by the bidirectional RRT* in an obstacle-strewn environment and Figure 7b shows the path planned by the improved bidirectional RRT*. By comparing Figure 7a,b, it is evident that the improved bi-directional RRT* algorithm plans shorter paths with fewer sampling points in the presence of obstacles, which is a significant improvement compared to the traditional bidirectional RRT* algorithm.

Comparing the two algorithms in the presence and absence of obstacles, the path length as well as the direction of the random tree's expansion and the number of sampling points show that the improved bidirectional RRT* algorithm plans shorter and more biased paths when expanded.

3.3. Path Optimization

There are still redundant nodes and redundant sections in the path planned by the improved bidirectional RRT*. A diagram of the redundant nodes is shown in Figure 8.

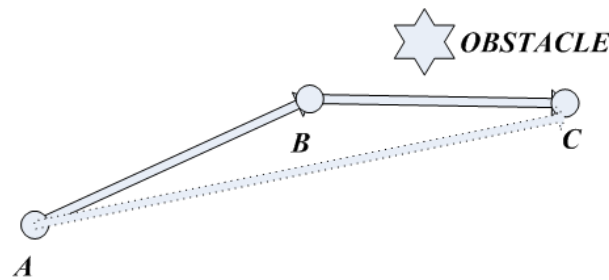


Figure 8. Diagram of the redundant nodes.

In Figure 8, the line between points A and C does not pass through the obstacle, in which case point B is the redundant node.

At this time, the path planned by the improved bidirectional RRT* algorithm is optimized to extract key nodes and eliminate redundant nodes. The specific steps of the process are as follows:

1. Put all the nodes into the set $\{P_1, P_2, P_3 \dots P_n\}$ in order.
2. Connect the nodes in the set one by one from the starting node P_t until the connection between the node with P_{t+1} passes the obstacle and P_t is the key point in the set. At this point, starting from P_t , connect the remaining nodes in turn until all the key points are found.
3. Connect the key points and target points in sequence from the starting point to plan the new path, as shown in Figure 8.

Figure 9b shows the key points on the path planned by the improved bidirectional RRT* in an obstacle-strewn environment and Figure 9c shows the re-routed path according to the key nodes.

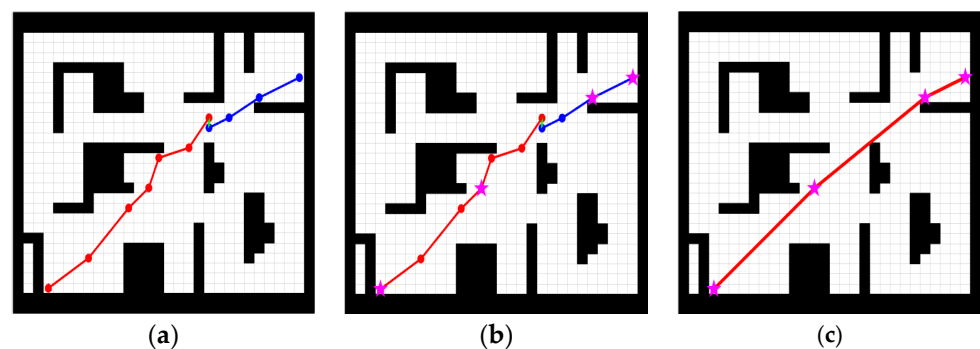


Figure 9. Optimization steps. (a) Original path, (b) Extraction of key nodes, (c) Final Path.

The formula for calculating the length of the path is as follows:

$$L = \sqrt[2]{|x_1 - x_2|^2 + |y_2 - y_2|^2} \quad (4)$$

Keep accumulating the distance between each of the two nodes until finally obtaining the length of the whole path.

As can be seen in Table 1, the optimization procedure has resulted in a 7.47% reduction in path length. There is a 77.8% reduction in the number of inflection points in the path. The optimized path is smoother, with fewer inflection points and a shorter path length.

Table 1. Comparison of improved RRT algorithm and optimized path.

Path-Planning Algorithms	Path Length	Number of Path Inflection Points
Improved bidirectional RRT*	35.4754	9
After optimizing the path	32.8269	2

4. The Incorporation of the Dynamic Window Method

The dynamic window method simulates the path of a robot at all speeds in the velocity space [19,20]; scores the robot's orientation, movement speed, and the distance to obstacles over a certain period of time; and aggregates the scores according to a certain ratio to obtain the path with the highest total score. Naturally, the path with the highest score is the best path.

4.1. Robot Kinematic Models

Assuming that, over a short period, the robot moves in a straight line with an angular velocity of ω and a linear velocity of v during movement, the motion of the robot in Δt can be described as follows:

$$\begin{cases} x_{t+\Delta t} = x_t + v \Delta t \cos(\theta_t) \\ y_{t+\Delta t} = y_t + v \Delta t \sin(\theta_t) \\ \theta_{t+\Delta t} = \theta_t + \omega \Delta t \end{cases} \quad (5)$$

Using Equation (5) and the velocity information collected, the trajectory of the robot can be simulated over the next Δt time.

4.2. Velocity Sampling

There are a large number of angular and linear velocities in velocity space. At this point, the velocity should be constrained, accounting for the robot's motor situation and the current environment, as in Equation (6).

$$V_m = \{(v, \omega) | v \in [v_{min}, v_{max}], \omega \in [\omega_{min}, \omega_{max}]\} \quad (6)$$

Motor torque and other factors will limit the movement speed of the robot. In a real environment, the moving speed of the robot is as follows:

$$V_d = \{(v, \omega) | v \in [v_c - \dot{v}_b \Delta t, v_c + \dot{v}_a \Delta t], \omega \in [\omega_c - \dot{\omega}_b \Delta t, \omega_c + \dot{\omega}_a \Delta t]\} \quad (7)$$

In Equation (7): the current velocity is v_c, ω_c ; the maximum acceleration is $\dot{v}_a, \dot{\omega}_a$; and the maximum deceleration is $\dot{v}_b, \dot{\omega}_b$.

To enable the robot to stop in time before it hits an obstacle, set an upper speed limit for the robot at:

$$v_a = \left\{ (v, \omega) \left| v \leq (2dist(v, \omega)\dot{v}_b)^{\frac{1}{2}}, \omega \leq (2dist(v, \omega)\dot{\omega}_b)^{\frac{1}{2}} \right. \right\} \quad (8)$$

In Equation (8), $dist(v, \omega)$ is the shortest distance between the simulated path and the obstacles. There are multiple sets of feasible trajectories in the sampled velocities, and it is necessary to use an evaluation function to select the optimal path.

$$G(v, \omega) = \sigma(\alpha head(v, \omega) + \beta dist(v, \omega) + \gamma vel(v, \omega)) \quad (9)$$

In Equation (9), $head(v, \omega)$ is used to evaluate the deviation of the target direction and the simulated trajectory direction at the current velocity. $vel(v, \omega)$ is used to evaluate the linear velocity of the robot. σ is the smoothing function and α , β , and γ are the coefficients.

The traditional dynamic window method can fall into a local optimum and thus fail to find a valid path in operation, as shown in Figure 10.

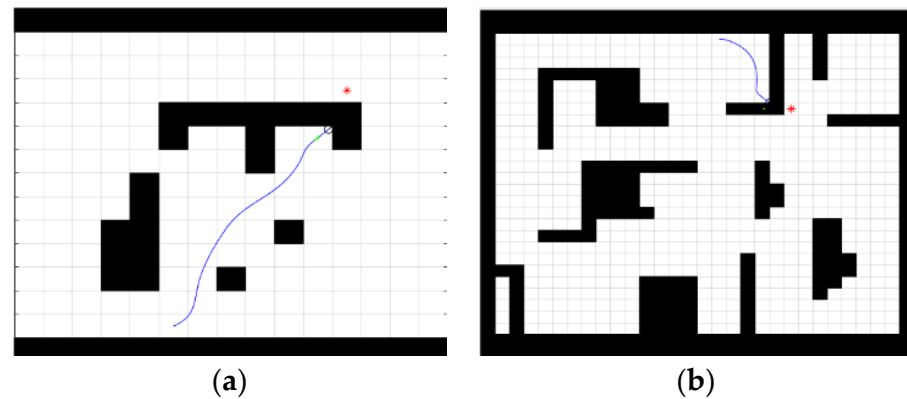


Figure 10. Traditional dynamic window method-based planning path. (a) 15 * 15 map, (b) 30 * 30 map.

As can be distinct seen in Figure 10a,b, the dynamic window method easily falls into a local optimum when there are corners in the ground, thus failing to find a globally valid path, so, in practice, the dynamic window method is used as a local algorithm. The improved bi-directional RRT* algorithm is used as a global algorithm combined with the local algorithm.

5. Fusion Algorithm

The traditional dynamic window method is prone to fall into local optimality when planning paths and requires a global path-planning algorithm to guide it. Global planning is performed using the improved RRT algorithm, which segments the global path according to key points and applies the dynamic window method to each segment of the path [21,22]. In the global algorithm, the improved RRT plans the global optimal path; in the local algorithm, the improved RRT algorithm guides the dynamic window method to plan the path, thereby preventing the dynamic window method from falling into a local optimum and failing to find the path. The schematic diagram of the fusion algorithm is shown in Figure 11.



Figure 11. Diagram of fusion algorithm.

In Figure 11, P1 and P2 are two adjacent key nodes, and the paths between the two critical nodes are planned using the dynamic window method. In order to understand the fusion algorithm flow more clearly, a flow chart of the fusion algorithm has been constructed, as shown in Figure 12.

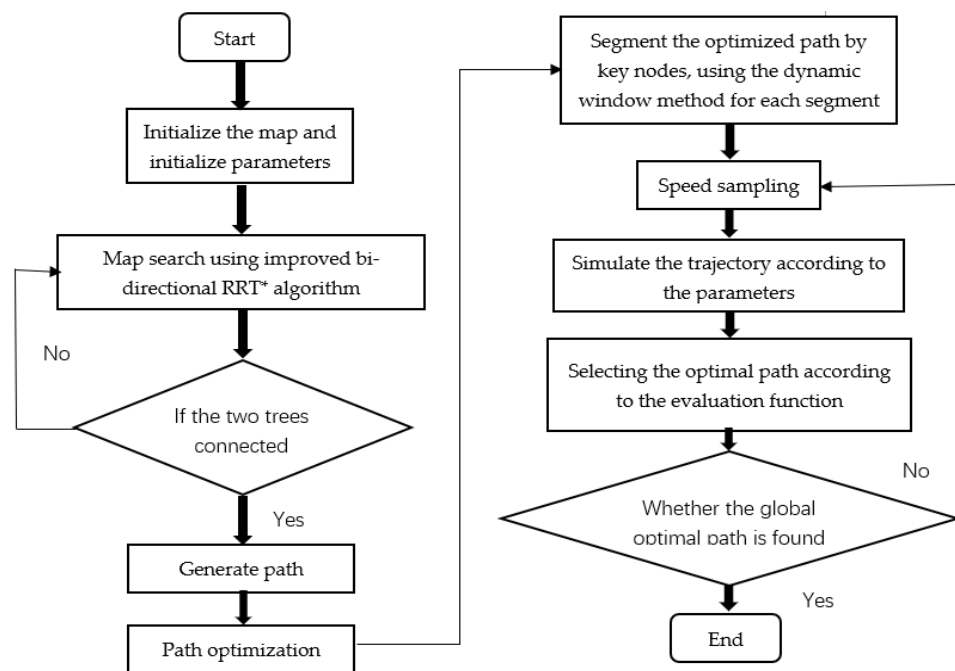


Figure 12. Flow chart of fusion algorithm.

6. Simulation Verification

To test the efficiency of the improved bidirectional RRT* algorithm and the effectiveness of the fusion algorithm, a matrix was used in MATLAB 2017 to establish a grid graph, with black squares representing obstacles and white squares representing clear space. The starting node is at (4,3); the target point is at (29,24). Two groups of experiments were used for detection and comparison.

Experiment 1: A comparison the path-planning performance of the, the traditional RRT, the traditional A*, the traditional bidirectional RRT*, the improved bidirectional RRT* algorithm, and the fusion algorithm is shown in Figure 13a–e.

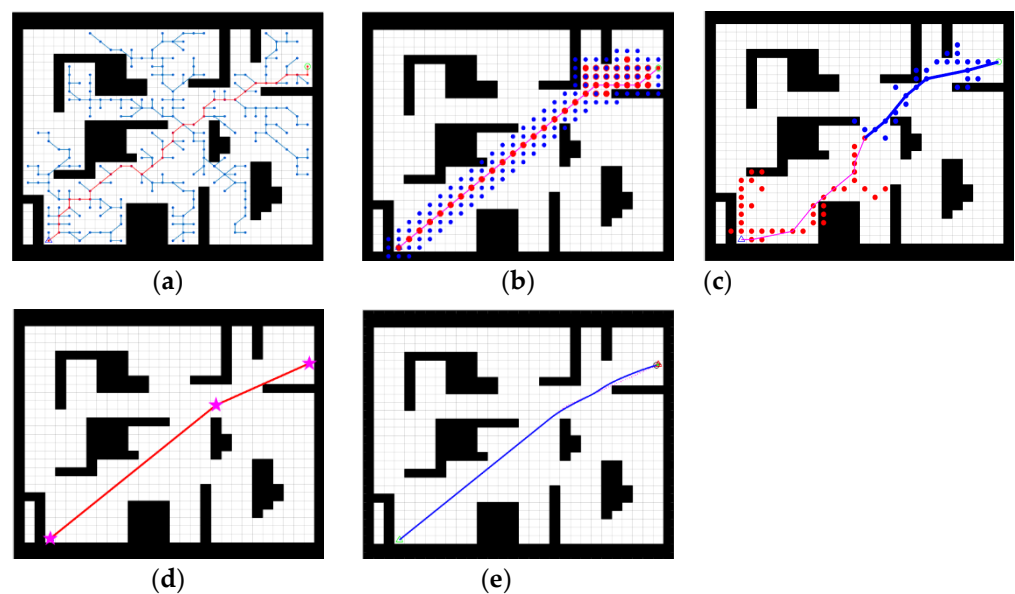


Figure 13. Five algorithms for planning the path. (a) Traditional RRT, (b) Traditional A*, (c) Traditional bidirectional RRT*, (d) Improved bidirectional RRT*, (e) Fusion algorithm.

Several algorithm extensions and sampling steps can be clearly seen in Figure 13. To better verify the effectiveness of the algorithm, a quantitative comparison was performed using the table. The five algorithms' plan times, path lengths, and number of inflection points are shown in Table 2.

Table 2. Comparison of five algorithms.

Path-Planning Algorithms	Path Length	Planning Time(s)	Number of Path Inflection Points
Traditional RRT	39.2132	1.3745	25
Traditional A*	33.6985	0.4971	2
Traditional bidirectional RRT*	35.0956	0.1178	10
Improved bidirectional RRT*	32.9230	0.0513	1
Fusion algorithm	32.7300	377.0923	none

In Table 2, these five algorithms are compared in terms of path length; it is evident that the fusion algorithm plans the shortest path length. The fusion algorithm reduces the path length by 0.59% compared to that of the improved bidirectional RRT* algorithm. In terms of path-planning time, the improved bidirectional RRT* algorithm uses the shortest time to plan an efficient path. In terms of path inflection points, except for the fusion algorithm, the improved bidirectional RRT* algorithm plans the path with the least number of inflection points and plans the smoothest path.

Experiment 2: Temporary obstacles were added to the global path planned by the improved bidirectional RRT* algorithm, as shown by the red squares in Figure 14, to test whether the fusion algorithm enables the robot to avoid temporary obstacles.

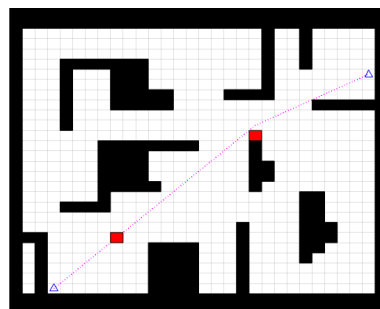


Figure 14. The addition of temporary obstacles to the global path.

By adding temporary obstacles to the global trajectory, the robot moves toward the target point as well as the obstacle avoidance state, as shown in Figure 15.

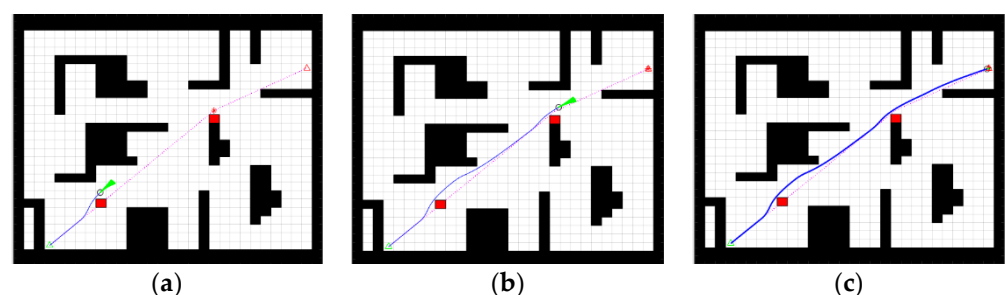


Figure 15. Overall trajectory. (a) First avoidance, (b) Second avoidance, (c) Reaching the target.

As shown in Figure 15, the path planned by the fusion algorithm effectively bypasses the two temporary obstacles and the path does not intersect with the obstacles in the figure, allowing the robot to reach the target point safely. This proves that the fusion algorithm is globally effective and that the planned path is able to avoid temporary obstacles.

Figure 16a shows the robot's linear movement speed and Figure 16b shows the angular velocity of the robot's movement. The robot reaches the target point safely, as evidenced by the velocity variation. Thus, the fusion algorithm has proven to be globally effective.

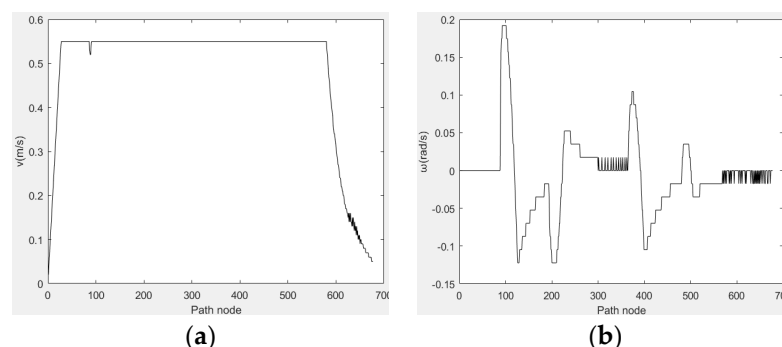


Figure 16. Speed variation in robot's movement. (a) Line Speed, (b) Angular velocity.

7. Conclusions

This paper offers two contributions. (1) The concept of the artificial potential field method was incorporated into the bidirectional RRT* algorithm to adjust the sampling points of random numbers generated at the starting point and the target point. Through simulation and comparison with the traditional A*, the traditional RRT, and the traditional RRT* algorithms, it has been verified that the improved bidirectional RRT* algorithm offers shorter path planning and higher path-planning efficiency. (2) The improved bi-directional RRT* algorithm was combined with the dynamic window method. It has been proven that the fusion algorithm can plan an effective path and avoid temporary obstacles. The algorithm also has some disadvantages, as its bias is more pronounced; therefore, it is not suitable for planning efficient paths in very complex environments.

The algorithm can be used in ROS mobile robots using the bidirectional RRT* algorithm as the global path algorithm. The path-planning algorithm will be explored in depth in future research for application in real robots.

Author Contributions: Conceptualization, P.X. and X.W.; methodology, X.L. and Y.W.; experiment, X.L., X.M. and Z.Z.; formal analysis, X.M. and X.W.; investigation, X.W.; writing—original draft preparation, P.X. and Y.W.; writing—review and editing, P.X. and X.M.; project administration, Y.W., Z.Z. and X.L. All authors have read and agreed to the published version of the manuscript.

Funding: The work in this paper thanks the support of the National Natural Science Foundations of China (No. 52001105) the Natural Science Foundation of Hebei Province (E2022402107), the University Science and Technology Research Project of Hebei Province (QN2021209 and BJ2021012), the Key projects of Hebei Provincial Department of Education (No. ZD2021024); the Handan science technology planning project (No. 21422301290).

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Panda, M.; Das, B.; Subudhi, B.; Pati, B.B. A Comprehensive Review of Path Planning Algorithms for Autonomous Underwater Vehicles. *Int. J. Autom. Comput.* **2020**, *17*, 321–352. [\[CrossRef\]](#)
2. Wang, X.; Yang, Y.; Wang, D.; Zhang, Z. Mission-oriented cooperative 3D path planning for modular solar-powered aircraft with energy optimization. *Chin. J. Aeronaut.* **2022**, *35*, 98–109. [\[CrossRef\]](#)
3. Chen, Y.; Bai, G.; Zhan, Y.; Hu, X.; Liu, J. Path Planning and Obstacle Avoiding of the USV Based on Improved ACO-APF Hybrid Algorithm with Adaptive Early-Warning. *IEEE Access* **2021**, *9*, 40728–40742. [\[CrossRef\]](#)
4. Mavrovouniotis, M.; Yang, S. Ant Colony Optimization with Immigrants Schemes in Dynamic Environments. *Int. Conf. Parallel Probl. Solving Nat.* **2010**, 6239, 371–380. [\[CrossRef\]](#)
5. Erke, S.; Bin, D.; Yiming, N.; Qi, Z.; Liang, X.; Dawei, Z. An improved A-Star based path planning algorithm for autonomous land vehicles. *Int. J. Adv. Robot. Syst.* **2020**, *17*, 172988142096226. [\[CrossRef\]](#)

6. Xie, L.; Henkel, C.; Stol, K.; Xu, W. Power-minimization and energy-reduction autonomous navigation of an omnidirectional Mecanum robot via the dynamic window approach local trajectory planning. *Int. J. Adv. Robot. Syst.* **2018**, *15*, 172988141875456. [\[CrossRef\]](#)
7. Yan, Z.; Hao, B.; Zhang, W.; Yang, S.X. Dubins-RRT Path Planning and Heading-Vector Control Guidance for a UAV Recovery. *Int. J. Robot. Autom.* **2016**, *31*, 251–262. [\[CrossRef\]](#)
8. Jeong, I.-B.; Lee, S.-J.; Kim, J.-H. Quick-RRT*: Triangular inequality-based implementation of RRT* with improved initial solution and convergence rate. *Expert Syst. Appl.* **2019**, *123*, 82–90. [\[CrossRef\]](#)
9. Mashayekhi, R.; Idris, M.Y.I.; Anisi, M.H.; Ahmady, I.; Ali, I. Informed RRT*-Connect: An Asymptotically Optimal Single-Query Path Planning Method. *IEEE Access* **2020**, *8*, 19842–19852. [\[CrossRef\]](#)
10. Wang, J.; Meng, M.Q.-H.; Khatib, O. EB-RRT: Optimal Motion Planning for Mobile Robots. *IEEE Trans. Autom. Sci. Eng.* **2020**, *17*, 2063–2073. [\[CrossRef\]](#)
11. Kang, J.-G.; Lim, D.-W.; Choi, Y.-S.; Jang, W.-J.; Jung, J.-W. Improved RRT-Connect Algorithm Based on Triangular Inequality for Robot Path Planning. *Sensors* **2021**, *21*, 333. [\[CrossRef\]](#)
12. Zhang, H.; Wang, Y.; Zheng, J.; Yu, J. Path Planning of Industrial Robot Based on Improved RRT Algorithm in Complex environments. *IEEE Access* **2018**, *6*, 53296–53306. [\[CrossRef\]](#)
13. Noreen, I.; Khan, A.; Ryu, H.; Doh, N.L.; Habib, Z. Optimal path planning in cluttered environment using RRT*-AB. *Intell. Serv. Robot.* **2018**, *11*, 41–52. [\[CrossRef\]](#)
14. Qi, J.; Yang, H.; Sun, H. MOD-RRT*: A Sampling-Based Algorithm for Robot Path Planning in Dynamic Environment. *IEEE Trans. Ind. Electron.* **2021**, *68*, 7244–7251. [\[CrossRef\]](#)
15. Wang, J.; Chi, W.; Li, C.; Wang, C.; Meng, M.Q.-H. Neural RRT*: Learning-Based Optimal Path Planning. *IEEE Trans. Autom. Sci. Eng.* **2020**, *17*, 1748–1758. [\[CrossRef\]](#)
16. Wang, W.; Zuo, L.; Xu, X. A Learning-based Multi-RRT Approach for Robot Path Planning in Narrow Passages. *J. Intell. Robot. Syst.* **2018**, *90*, 81–100. [\[CrossRef\]](#)
17. Qie, T.; Wang, W.; Yang, C.; Li, Y.; Liu, W.; Xiang, C. A path planning algorithm for autonomous flying vehicles in cross-country environments with a novel TF-RRT* method. *Green Energy Intell. Transp.* **2022**, *1*, 100026. [\[CrossRef\]](#)
18. Guo, J.; Xia, W.; Hu, X.; Ma, H. Feedback RRT* algorithm for UAV path planning in a hostile environment. *Comput. Ind. Eng.* **2022**, *174*, 108771. [\[CrossRef\]](#)
19. Yang, W.; Wu, P.; Zhou, X.; Lv, H.; Liu, X.; Zhang, G.; Hou, Z.; Wang, W. Improved Artificial Potential Field and Dynamic Window Method for Amphibious Robot Fish Path Planning. *Appl. Sci.* **2021**, *11*, 2114. [\[CrossRef\]](#)
20. Maroti, A.; Szaloki, D.; Kiss, D.; Tevesz, G. Investigation of Dynamic Window based navigation algorithms on a real robot. In Proceedings of the 2013 IEEE 11th International Symposium on Applied Machine Intelligence and Informatics (SAMII), Herl'any, Slovakia, 31 January–2 February 2013; pp. 95–100. [\[CrossRef\]](#)
21. Liu, L.; Yao, J.; He, D.; Chen, J.; Huang, J.; Xu, H.; Wang, B.; Guo, J. Global Dynamic Path Planning Fusion Algorithm Combining Jump-A* Algorithm and Dynamic Window Approach. *IEEE Access* **2021**, *9*, 19632–19638. [\[CrossRef\]](#)
22. Zhong, X.; Tian, J.; Hu, H.; Peng, X. Hybrid Path Planning Based on Safe A* Algorithm and Adaptive Window Approach for Mobile Robot in Large-Scale Dynamic Environment. *J. Intell. Robot. Syst.* **2020**, *99*, 65–77. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.